

IF29 — Data Analytics

Comparaison de deux méthodes de classification

Rapport de projet

ARIDORY Malcolm

BEN ABDALLAH Mohamed

LANOUAR Mariem

RODRIGUES Gwendal

UTT — Printemps 2026

Contents

1. Introduction	4
1.1. Contexte	4
1.2. Problématique	4
1.3. Définition d'un profil atypique	4
1.4. Structure du rapport	5
2. Description des données	6
2.1. Source et format	6
2.2. Volumétrie	6
2.3. Choix du système de stockage	6
3. Pipeline ETL (Extract - Transform - Load)	7
3.1. Architecture générale	7
3.2. Extraction : ingestion des tweets (<code>ingest.py</code>)	7
3.3. Transformation : déduplication des utilisateurs (<code>extract_users.py</code>)	8
3.4. Chargement : upsert vers la table <code>users</code> (<code>load_users.py</code>)	9
3.5. Indexation pour la performance	9
3.6. Validation expérimentale : comparaison MongoDB vs PostgreSQL	9
3.6.1. Performance d'ingestion	9
3.6.2. Performance de calcul des features	10
3.6.3. Performance de récupération (fetch)	10
3.6.4. Synthèse	10
3.7. Structure finale de la table <code>users</code>	11
4. Labellisation manuelle des profils	12
4.1. Méthodologie d'échantillonnage	12
4.2. Fichier d'annotation	12
4.3. Critères de labellisation	13
4.3.1. Indicateurs de profil (User-based features)	13
4.3.2. Indicateurs d'activité (Activity-based features)	14
4.3.3. Indicateurs de contenu (Content-based features)	14
4.4. Processus de labellisation	14
5. Analyse exploratoire des données (EDA)	16
5.1. Distribution des classes	16
5.2. Analyse bivariée : Graphe social et Engagement	17
5.3. Personnalisation du profil	17
5.4. Matrice de corrélation	18
5.5. Synthèse dimensionnelle (Scores SPOT)	19
6. Méthodologie de modélisation	20
6.1. Feature engineering	20
6.2. Préparation des données pour l'apprentissage	21
6.2.1. Nettoyage et exclusions	21
6.2.2. Traitement de la colinéarité	22
6.2.3. Choix du type de normalisation	22
6.3. Modélisation Supervisée : Random Forest	22

6.3.1. Échantillonnage et validation	22
6.3.2. Optimisation des hyperparamètres	23
6.3.3. Importance des features	23
6.3.4. Évaluation et Matrice de Confusion	24
6.4. Modélisation Supervisée : SVM	25
6.4.1. Échantillonnage et validation	25
6.4.2. Réduction dimensionnelle	25
6.4.3. Optimisation des hyperparamètres	26
6.4.4. Importance des features	27
6.4.5. Évaluation	27
6.5. Conclusion	29
6.5.1. interprétation des résultats	29
6.5.2. Point de vigilance	29
6.5.3. Pour conclure	29
7. Modélisation Non-Supervisée	30
7.1. MiniBatch K-means sur l'ensemble des profils	30
7.2. Clustering spectral sur les 420 profils	31
8. Discussion, résultats et conclusion	32
8.1. Bilan des modèles	32
8.2. Random Forest contre clustering	32
8.3. Limites	32
8.4. Ouverture : POC sur la répétition sémantique	33
8.5. Conclusion	34
9. Bibliographie	35

1. Introduction

1.1. Contexte

Twitter (aujourd'hui X) est l'une des principales plateformes de microblogging au monde, avec plusieurs centaines de millions d'utilisateurs actifs générant un flux continu de messages courts appelés **tweets**. Cette masse de données, accessible via des APIs publiques, constitue un terrain d'analyse privilégié pour les sciences des données, qu'il s'agisse d'étudier des phénomènes sociaux, politiques ou informationnels.

Cependant, tous les profils présents sur la plateforme ne sont pas animés par des humains. Une part significative de l'activité est générée par des comptes automatisés, appelés **bots**, dont les objectifs peuvent varier de la simple diffusion d'informations (bots météorologiques, alertes sismiques) à des usages malveillants (spam, phishing, manipulation de l'opinion, désinformation). La littérature scientifique s'est emparée de cette problématique depuis le milieu des années 2010, avec des travaux fondateurs comme ceux de Ferrara et al. (2016) sur la montée des bots sociaux, de Chu et al. (2012) sur la distinction entre humain, bot et cyborg, ou encore de Varol et al. (2017) sur la détection et la caractérisation des interactions humain-bot.

1.2. Problématique

Ce projet s'inscrit dans cette continuité. Il vise à répondre à la question suivante : **comment distinguer automatiquement un profil humain d'un profil atypique (bot, spammeur, compte automatisé) à partir des données publiques de Twitter ?**

Plus spécifiquement, nous cherchons à comparer deux paradigmes de classification :

- Une **approche supervisée** : entraîner un modèle sur un échantillon de profils préalablement labellisés manuellement (humain / bot).
- Une **approche non-supervisée** : laisser un algorithme de clustering identifier des groupes naturels dans les données, puis interpréter ces groupes **a posteriori**.

Le jeu de données **Tweet_Worldcup.zip** contient des tweets collectés pendant la Coupe du Monde de football 2018 (juin-juillet 2018). Cet événement planétaire a généré un volume exceptionnel d'activité sur Twitter – des millions de tweets par jour avec des pics lors des matchs, des buts et des controverses – ce qui en fait un terrain d'étude idéal pour la détection de comportements automatisés à grande échelle.

1.3. Définition d'un profil atypique

Dans le cadre de ce projet, nous définissons un **profil atypique** comme un compte dont les caractéristiques comportementales et statistiques s'écartent significativement de celles de la population moyenne des utilisateurs. Cette définition, volontairement large, recouvre plusieurs réalités :

- **Les bots** : comptes entièrement ou partiellement automatisés, caractérisés par une fréquence de publication anormalement élevée, un ratio followers/followees déséquilibré, et une faible personnalisation du profil (Ferrara et al., 2016 ; Chu et al., 2012).
- **Les spammeurs** : comptes diffusant massivement des URLs, souvent raccourcies, dans un but publicitaire ou malveillant (Benevenuto et al., 2010).

- **Les profils de désinformation** : comptes créés pour amplifier artificiellement certains messages ou simuler un soutien populaire (astroturfing).

Dans le cadre de ce projet, nous nous concentrons sur la détection des bots et des spammeurs. Bien que d'autres catégories de profils atypiques existent (cyborgs, trolls rémunérés, faux comptes de promotion), les caractéristiques de notre jeu de données prêtent particulièrement bien à l'identification des comptes automatisés et des diffuseurs massifs de liens. Les profils de désinformation (astroturfing) nécessiteraient une analyse avancée du réseau social et du contenu qui dépasse le périmètre de ce projet.

1.4. Structure du rapport

Ce rapport suit le déroulement classique d'un projet de data science, en reprenant avec souplesse une approche semblable à celle du framework CRISP-DM : nous avons la compréhension métier, nous poursuivrons avec l'analyse des données, la préparation de notre base de données et la pipeline ETL, les modélisations (supervisée et non-supervisée) et enfin l'évaluation et la mise en production de nos modèles.

2. Description des données

2.1. Source et format

Le jeu de données **Tweet_Worldcup.zip** est constitué de fichiers JSON contenant des tweets collectés pendant la Coupe du Monde de football 2018. Chaque ligne de chaque fichier correspond à un tweet au format JSON tel que renvoyé par l'API Twitter v1.

Un tweet typique contient :

- Les **métadonnées du tweet** : identifiant (`id`), date de création (`created_at`), texte (`full_text` ou `text`), langue (`lang`).
- Les **entités** mentionnées dans le tweet : hashtags (`#`), mentions (`@`), URLs, médias attachés.
- L'**objet utilisateur (user object)** imbriqué : identifiant (`user.id`), pseudonyme (`screen_name`), nom affiché (`name`), description, nombre de followers, nombre d'amis (**following**), nombre de statuts postés, statut de vérification, date de création du compte, etc.

Le fait que l'objet utilisateur soit **imbriqué** dans chaque tweet est une caractéristique importante du schéma de données : un même utilisateur apparaît autant de fois qu'il a de tweets dans le corpus, avec potentiellement des valeurs différentes de followers, friends, etc. selon la date du tweet (instantané du profil à l'instant de la collecte).

2.2. Volumétrie

Après ingestion complète, notre base de données PostgreSQL contient environ **4,5 millions de tweets** collectés pendant la Coupe du Monde 2018 (juin–juillet 2018), émis par plus de **1,8 million d'utilisateurs distincts**. La taille totale de la base de données PostgreSQL avoisine les **30 Go**, dont l'essentiel est occupé par la colonne `raw_data` en JSONB qui stocke l'intégralité des objets tweets.

2.3. Choix du système de stockage

Le sujet du projet suggère MongoDB pour le stockage des données semi-structurées. Nous avons fait le choix alternatif de **PostgreSQL** pour les raisons suivantes : la **maturité de l'écosystème SQL** pour l'agrégation et l'analyse exploratoire (fenêtrage, agrégats, jointures) ; le support natif du type **JSONB**, qui permet de conserver les données brutes dans leur format d'origine tout en offrant des capacités d'indexation et de requêtage performantes via les expressions de chemin JSON ; et la **compatibilité avec les outils Python** de data science (psycopg2, pandas).

Ce choix constitue un compromis entre la flexibilité du NoSQL et la puissance du relationnel, particulièrement adapté à un projet mêlant données brutes (JSONB) et données structurées (tables relationnelles). Notre validation expérimentale (section 3) confirme que PostgreSQL est deux fois plus rapide que MongoDB pour la récupération des tweets, tout en offrant des performances de calcul de features comparables après dénormalisation.

3. Pipeline ETL (Extract - Transform - Load)

Cette section décrit l'architecture du pipeline de traitement des données, depuis l'ingestion des fichiers JSON bruts jusqu'à la constitution d'une table `users` exploitable pour la classification.

3.1. Architecture générale

Le pipeline s'articule autour de deux tables PostgreSQL :

- **tweets** : stocke les tweets bruts dans leur intégralité au format JSONB. Un index d'expression est créé sur l'identifiant utilisateur extrait du JSON pour permettre des requêtes performantes par utilisateur.
- **users** : table relationnelle contenant un enregistrement par utilisateur distinct, avec le snapshot utilisateur le plus récent (dernier tweet connu).

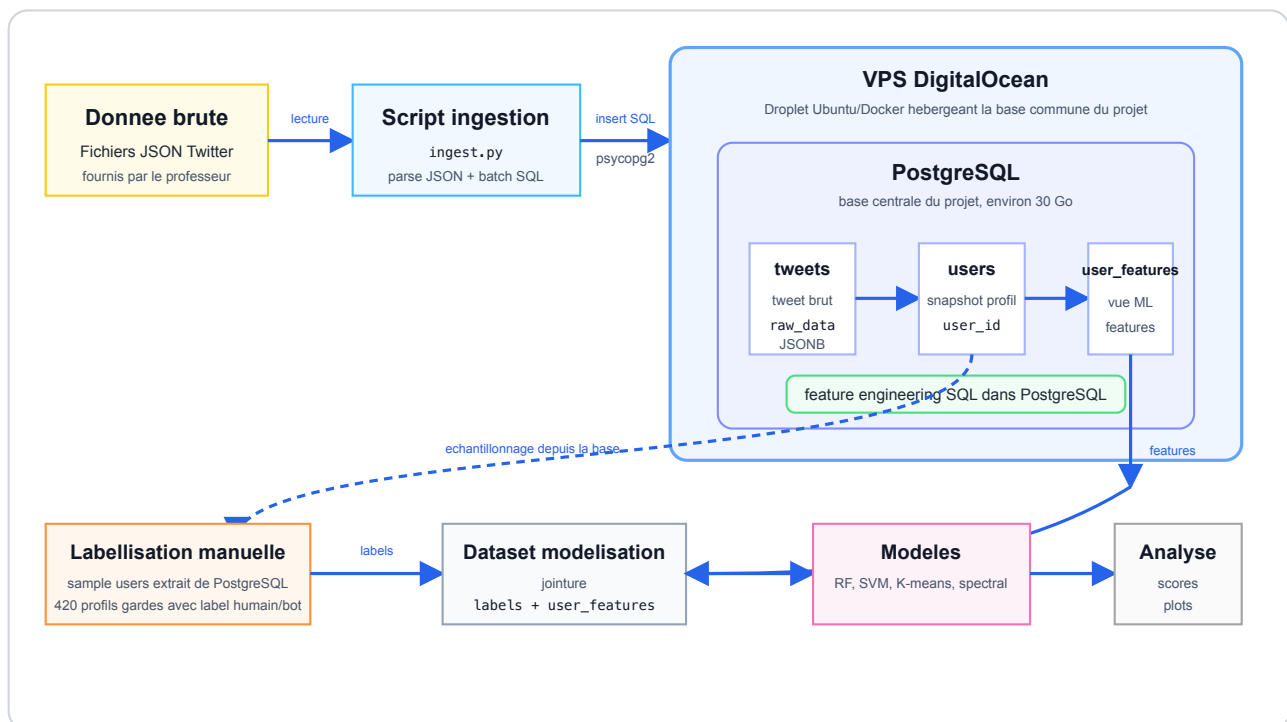


Figure 1: Architecture de dépendances : donnée brute, ingestion, VPS PostgreSQL et modélisation

Ce schéma résume le choix principal du projet : la donnée brute est lue par un script d'ingestion, puis envoyée vers PostgreSQL, hébergé sur le VPS DigitalOcean. Les transformations utiles au ML partent ensuite de cette base : `tweets` sert à construire `users`, puis `user_features`. Les modèles ne consomment donc pas directement les fichiers JSON : ils utilisent les features calculées depuis PostgreSQL et jointes aux labels.

3.2. Extraction : ingestion des tweets (`ingest.py`)

Le script `src/etl/extract/ingest.py` est responsable du chargement initial des fichiers JSON dans la table `tweets`. Son fonctionnement est le suivant :

1. Il lit récursivement tous les fichiers `.json` du répertoire `data/raw/`.
2. Pour chaque fichier, chaque ligne est parsée comme un objet JSON.

3. Les champs `id`, `created_at` et `text` (ou `full_text`) sont extraits et stockés dans des colonnes SQL typées ; l'intégralité du JSON est conservée dans la colonne `raw_data` de type `JSONB`.
4. L'insertion se fait par lots (**batch**) de 2 000 tweets via `execute_values` de `psycopg2`, avec une clause `ON CONFLICT (id) DO NOTHING` pour garantir l'idempotence (un fichier déjà ingéré ne duplique pas les données).
5. Un fichier `processed_files.log` assure le suivi des fichiers déjà traités, permettant de reprendre l'ingestion après une interruption sans repartir de zéro.

L'ingestion complète des fichiers JSON s'est déroulée sans erreur bloquante. Quelques lignes mal formatées ont été rencontrées et ignorées silencieusement: elles ne concernaient qu'un fichier et ont été ignorées. Le temps d'ingestion total pour l'ensemble du corpus (plusieurs centaines de fichiers) est d'environ 20 à 30 minutes. Sur un sous-ensemble de 50 fichiers (100 000 tweets), l'ingestion PostgreSQL prend exactement 107 secondes.

La structure de la table `tweets` est la suivante :

Colonne	Type	Description
<code>id</code>	<code>BIGINT</code>	Identifiant unique du tweet (PK)
<code>created_at</code>	<code>TIMESTAMP</code>	Date de création du tweet
<code>tweet_text</code>	<code>TEXT</code>	Texte du tweet
<code>raw_data</code>	<code>JSONB</code>	Objet JSON complet du tweet

3.3. Transformation : déduplication des utilisateurs (`extract_users.py`)

Le cœur de la transformation réside dans le script `src/etl/transform/extract_users.py`.

Problème. La table `tweets` peut contenir plusieurs occurrences du même utilisateur, chacune avec un snapshot potentiellement différent (le nombre de followers évolue dans le temps). Il est nécessaire de choisir quelle version retenir.

Solution retenue : snapshot le plus récent. Nous utilisons une clause `DISTINCT ON` de PostgreSQL triée par `user_id`, `created_at DESC`. Cela garantit que, pour chaque utilisateur, seul le tweet le plus récent est conservé — et donc la version la plus à jour de son profil **au moment de la collecte**. Cette approche est préférable à une moyenne (le nombre de followers n'est pas une mesure qu'on lisse) et à un snapshot aléatoire (non déterministe).

La requête SQL sous-jacente est :

```
SELECT DISTINCT ON ((raw_data->'user'->'id')::BIGINT)
    (raw_data->'user'->'id')::BIGINT AS user_id,
    raw_data->'user'                AS raw_user,
    created_at                      AS tweet_created_at
FROM tweets
WHERE raw_data->'user'->'id' IS NOT NULL
ORDER BY (raw_data->'user'->'id')::BIGINT, created_at DESC
```

Une deuxième passe agrège par utilisateur les statistiques `tweet_count`, `first_seen_at` et `last_seen_at` (premier et dernier tweet dans notre corpus).

L'extraction des utilisateurs distincts depuis la table `tweets` utilise un curseur côté serveur (server-side cursor) avec une taille de lot de 5 000 lignes, ce qui évite de charger les 4.5 millions de

lignes en mémoire d'un seul coup. Sur le sous-ensemble de 50 fichiers (100 000 tweets), l'extraction des 74 779 utilisateurs uniques prend environ 4.4 secondes.

3.4. Chargement : upsert vers la table `users` (`load_users.py`)

Le script `src/etl/load/load_users.py` orchestre l'ensemble du pipeline :

1. Il applique `init.sql` (création de la table `users` et des index d'expression) si ce n'est pas déjà fait.
2. Il appelle `extract_users()` pour obtenir la liste des utilisateurs uniques.
3. Il insère les utilisateurs dans la table `users` par lots de 50 000, avec une clause `ON CONFLICT (user_id) DO UPDATE` (upsert) qui garantit l'idempotence.

Un mécanisme de **checkpoint** (fichier pickle) sauvegarde la liste extraite avant l'upsert. En cas d'échec, le prochain lancement recharge le pickle et saute l'extraction, évitant ainsi de perdre 25 minutes de calcul.

Sur le corpus complet, cette étape alimente la table `users` avec plus de **1,8 million d'utilisateurs distincts**. Le point le plus coûteux n'est pas l'insertion SQL elle-même mais la traversée des 4,5 millions de tweets, l'extraction des objets `user` imbriqués et leur sérialisation en `JSONB`. Le chargement par lots de 50 000 lignes limite la taille des transactions PostgreSQL et rend le processus reprenable : en pratique, le checkpoint évite de relancer une extraction d'environ 25 minutes en cas d'erreur pendant l'upsert.

3.5. Indexation pour la performance

Pour éviter des parcours complets de la table `tweets` à chaque requête par utilisateur, nous avons créé deux index d'expression :

```
CREATE INDEX idx_tweets_user_id
ON tweets (((raw_data->'user'->'id')::BIGINT));
```

```
CREATE INDEX idx_tweets_user_date
ON tweets (((raw_data->'user'->'id')::BIGINT), created_at DESC);
```

Ces index permettent à PostgreSQL d'utiliser des parcours d'index (index scan) plutôt que des parcours séquentiels (sequential scan) pour les requêtes filtrant ou triant par utilisateur. Le choix d'un index d'expression plutôt que d'une colonne générée (`GENERATED ALWAYS AS ... STORED`) est motivé par le souci de ne pas réécrire la table `tweets` qui pèse plusieurs dizaines de gigaoctets.

3.6. Validation expérimentale : comparaison MongoDB vs PostgreSQL

Afin de valider objectivement notre choix de système de stockage, nous avons implémenté une pipeline identique avec MongoDB (v7.0) et comparé les performances sur les trois étapes critiques : ingestion, calcul des features utilisateur et fetch des utilisateurs enrichis. Les tests ont été réalisés sur un sous-ensemble croissant de fichiers (1, 20 et 50 fichiers, soit jusqu'à 100 000 tweets) avec des lots d'insertion de 2 000 documents (1 insertion par fichier).

3.6.1. Performance d'ingestion

Fichiers	Tweets	PostgreSQL	MongoDB
1	2 000	2,02 s	0,35 s
20	40 000	40,67 s	6,81 s

Fichiers	Tweets	PostgreSQL	MongoDB
50	100 000	106,74 s	19,19 s

MongoDB est environ 5-6 fois plus rapide que PostgreSQL pour l'ingestion brute. Cet écart s'explique par l'absence de contraintes relationnelles, de typage des colonnes et de journalisation ACID complète dans MongoDB, chaque tweet est inséré tel quel sous forme de document BSON sans validation de schéma.

3.6.2. Performance de calcul des features

Pour le calcul des features sur les 74 779 utilisateurs extraits de 50 fichiers :

Système	Temps
MongoDB (agrégation pipeline)	14,58 s
PostgreSQL JSONB (materialized view)	33,96 s
PostgreSQL relationnel (materialized view)	14,21 s

Les performances sont comparables entre MongoDB et PostgreSQL en mode relationnel. La version JSONB de PostgreSQL est environ 2 fois plus lente, ce qui confirme l'intérêt de dénormaliser les données utilisateur dans une table relationnelle dédiée plutôt que de travailler directement sur le JSONB brut.

3.6.3. Performance de récupération (fetch)

Nous avons mesuré le temps de récupération des utilisateurs enrichis, répétée plusieurs fois pour stabiliser la mesure :

Users récupérés	PostgreSQL (JSONB)	PostgreSQL (relationnel)	MongoDB
5 000	90 ms	109 ms	148 ms
20 000	363 ms	434 ms	653 ms
100 000	1 376 ms	1 401 ms	3 123 ms

PostgreSQL est systématiquement **1,5 à 2 fois plus rapide** que MongoDB pour la récupération de tweets, que la requête utilise la colonne JSONB ou une table relationnelle. Cet avantage provient de la maturité des index B-tree de PostgreSQL et de l'optimisation des parcours d'index pour les accès ponctuels.

3.6.4. Synthèse

Ces résultats valident notre architecture : PostgreSQL pour le stockage et l'interrogation structurée (récupération rapide, requêtes analytiques via SQL), avec une vue matérialisée `user_features` pour les calculs de features. MongoDB conserve un avantage pour l'ingestion pure, mais la rapidité de récupération et la puissance du SQL pour l'analyse exploratoire justifient complètement à notre sens le choix de PostgreSQL pour ce projet.

3.7. Structure finale de la table users

Colonne	Type	Description
user_id	BIGINT	Identifiant Twitter du compte (PK)
screen_name	TEXT	Pseudonyme
name	TEXT	Nom affiché
description	TEXT	Bio du profil
location	TEXT	Localisation déclarée
url	TEXT	URL dans le profil
followers_count	INT	Nombre d'abonnés
friends_count	INT	Nombre d'abonnements (following)
listed_count	INT	Nombre de listes incluant ce compte
favourites_count	INT	Nombre de likes émis
statuses_count	INT	Nombre total de tweets postés
verified	BOOLEAN	Compte certifié par Twitter
created_at	TIMESTAMP	Date de création du compte
profile_image_url	TEXT	URL de la photo de profil
default_profile	BOOLEAN	Profil jamais personnalisé
default_profile_image	BOOLEAN	Photo de profil par défaut
raw_user	JSONB	Objet utilisateur complet (JSON)
tweet_count	INT	Tweets de cet utilisateur dans le corpus
last_tweet_at	TIMESTAMP	Date du tweet le plus récent
first_seen_at	TIMESTAMP	Date du premier tweet capturé
last_seen_at	TIMESTAMP	Date du dernier tweet capturé

4. Labellisation manuelle des profils

L'approche supervisée de classification nécessite un jeu de données d'entraînement étiqueté. En l'absence de **ground truth** (les données Twitter ne contiennent pas de label « bot » ou « humain »), nous avons constitué manuellement un corpus annoté.

4.1. Méthodologie d'échantillonnage

Le script `src/ml/sample_users.py` extrait un échantillon aléatoire de 1 000 utilisateurs depuis la table `users`. La sélection est **déterministe et reproductible** : elle utilise un hachage MD5 combinant l'identifiant utilisateur et une graine fixe (*seed*), ce qui garantit que la même graine produit toujours le même échantillon, indépendamment de la version de PostgreSQL ou de l'état du générateur aléatoire :

```
SELECT user_id FROM users
ORDER BY MD5(user_id::text || 'if29')
LIMIT 1000
```

4.2. Fichier d'annotation

Pour chaque utilisateur, le CSV exporté contient : les **métadonnées du profil** (20 colonnes issues de la table `users`) ; des **features calculées** aidant à la décision (ratio followers/friends, tweets par jour depuis la création du compte, tweets par jour dans le corpus, durée de couverture dans le corpus) ; les **5 tweets les plus récents** de l'utilisateur dans notre corpus, concaténés dans une colonne `sample_tweets` ; et deux colonnes vides à remplir : `label` (0 = humain, 1 = bot) et `notes`.

Le tableau ci-dessous détaille les indicateurs clés mis à disposition de l'annotateur.

Colonne	Signification	Utilisation
<code>followers_count</code>	Nombre d'abonnés	Un compte avec 0 followers et beaucoup de tweets est suspect
<code>friends_count</code>	Abonnements (following)	Un ratio followers/friends très déséquilibré (>5) suggère un bot
<code>listed_count</code>	Listes incluant le compte	Un compte jamais listé (<2) est peu visible, potentiellement automatisé
<code>statuses_count</code>	Tweets totaux postés	Un compte récent avec >10k statuts indique une automatisation
<code>favourites_count</code>	Likes émis	Un bot like souvent mécaniquement ; un humain a un ratio likes/tweets plausible
<code>verified</code>	Compte certifié (blue check)	Un compte vérifié est quasiment certainement humain (ou organisation légitime)

Colonne	Signification	Utilisation
account_created_at	Date de création du compte	Un compte créé récemment mais déjà très actif est suspect
default_profile	Profil jamais personnalisé	Laissé à true = aucune customisation = indice de bot
default_profile_image	Photo par défaut (oeuf)	Idem, absence d'avatar personnalisé
followers_friends_ratio	Ratio friends/followers	>1 = suit plus qu'il n'est suivi ; >5 = fort soupçon de bot
tweets_per_day_since_creation	Tweets/jour depuis création	>50 tweets/jour en moyenne est humainement impossible en continu
dataset_coverage_days	Jours de présence dans le corpus	Permet de contextualiser le tweet_count
tweets_per_day_in_dataset	Tweets/jour dans le corpus	Même interprétation que ci-dessus, limité à notre fenêtre d'observation
sample_tweets	5 derniers tweets	Lecture du contenu : le texte est-il varié ? naturel ? dupliqué ?

4.3. Critères de labellisation

Nous avons adopté une grille d'analyse fondée sur la littérature scientifique, en nous appuyant notamment sur les travaux de Ferrara et al. (2016), Chu et al. (2012) et Varol et al. (2017), ainsi que sur le framework SPOT de Perez et al. (2011). Les critères sont classés en trois catégories.

4.3.1. Indicateurs de profil (User-based features)

Ces indicateurs portent sur les métadonnées du compte, indépendamment de son activité récente. Un profil atypique présente typiquement :

- Une **absence de personnalisation** : `default_profile = true` et `default_profile_image = true` simultanément signalent un compte généré automatiquement sans effort de personnalisation humaine.
- Un **déséquilibre du graphe social** : un ratio `friends / followers` supérieur à 5 indique un compte qui suit massivement sans être suivi en retour — comportement caractéristique des bots de type « follow spam ».
- Une **date de création récente** couplée à un volume de statuts anormalement élevé (ex. compte créé il y a 3 mois avec 50 000 tweets).

4.3.2. Indicateurs d'activité (Activity-based features)

Ces indicateurs sont calculés à partir de l'activité observée du compte :

- Une **fréquence de publication excessive** : un rythme soutenu de plus de 50 tweets par jour, maintenu sur une longue période, dépasse les capacités humaines. Le papier SPOT (Perez et al., 2011) qualifie ce phénomène de **degré d'agressivité**.
- Une **présence massive d'URLs** dans les tweets : les bots diffusent souvent des liens raccourcis à des fins de spam ou de phishing.

4.3.3. Indicateurs de contenu (Content-based features)

L'analyse du texte des tweets (`sample_tweets`) permet de trancher les cas ambigus :

- **Duplication de contenu** : des tweets quasi-identiques postés à intervalles réguliers indiquent une automation par template.
- **Cohérence sémantique** : un humain produit des tweets variés, contextuels, parfois avec des fautes de frappe, des émotions, des réactions à d'autres utilisateurs.
- **Présence de hashtags et mentions** : un usage excessif et systématique de # et @ dans chaque tweet est un marqueur d'automation (visibilité optimisée dans le cadre SPOT).

4.4. Processus de labellisation

Afin de labelliser les différents profils, nous avons ouvert le fichier CSV dans un tableur, et nous avons déterminé par inspection visuelle, en prenant en compte les différentes métriques calculées, si le profil que nous avons sous les yeux nous semblait être celui d'un bot ou non.

The screenshot shows a Numbers spreadsheet titled 'labelling_sample'. The spreadsheet contains a table with columns for various metrics and a manual 'label' column. The columns are: followers_count, friends_count, listed_count, favourites_count, statuses_count, verified, account_created_at, default_profile, default_profile_image, tweet_count_in_dataset, first_seen_at, last_seen_at, followers_friends_ratio, tweets_per_day_since_creation, and label. The data is organized into rows, with some rows highlighted in green. The 'label' column contains values like 'True' and 'False', indicating whether a profile is a bot or not. The spreadsheet also shows a sidebar with formatting options and a status bar at the bottom.

	followers_count	friends_count	listed_count	favourites_count	statuses_count	verified	account_created_at	default_profile	default_profile_image	tweet_count_in_dataset	first_seen_at	last_seen_at	followers_friends_ratio	tweets_per_day_since_creation	label
1	553	1564	0	5293	2420	False	2011-06-16T13:10:52	True	False	18	2018-06-15T10:53:35	2018-06-17T15:41:55	2.8282	0.85	
2	697	611	8	24023	37893	False	2013-12-17T15:13:28	False	False	26	2018-06-14T07:50:34	2018-06-17T11:33:59	0.8766	23.08	
3	492	628	64	39238	210310	False	2013-05-31T13:57:29	True	False	17	2018-06-14T10:09:42	2018-06-16T14:57:29	1.2764	114.17	
4	5177	3018	7	5064	19229	False	2011-06-14T09:44:55	False	False	8	2018-06-14T17:19:23	2018-06-16T15:18:26	0.583	7.51	
5	2749	2149	14	7382	9021	False	2013-03-18T09:32:36	False	False	9	2018-06-15T12:22:35	2018-06-16T20:27:35	0.7817	4.71	
6	133	218	5	3693	6003	False	2014-10-19T07:44:57	False	False	8	2018-06-14T03:12:48	2018-06-16T01:32:54	1.6391	4.5	
7	11576	436	100	168490	79585	False	2012-01-12T12:59:23	False	False	22	2018-06-16T13:32:33	2018-06-17T16:43:48	0.0377	33.89	
8	1233	1831	9	5118	72754	False	2012-09-28T09:44:21	False	False	22	2018-06-14T02:38:24	2018-06-17T14:21:01	1.485	34.84	
9	595	165	37	53216	31186	False	2014-11-30T17:48:40	False	False	22	2018-06-14T09:57:54	2018-06-17T11:40:28	0.2773	24.1	
10	1009	4887	1	12036	15518	False	2012-01-22T19:50:34	True	False	13	2018-06-14T08:05:42	2018-06-16T16:10:57	4.8434	6.84	
11	242	432	9	2409	14460	False	2011-05-28T21:27:38	True	False	19	2018-06-14T08:12:03	2018-06-17T15:10:09	1.7851	5.81	
12	994	960	12	3796	28021	False	2009-07-31T20:08:13	False	False	6	2018-06-14T18:31:29	2018-06-15T18:30:16	0.9658	8.65	
13	1321	206	9	648	16259	False	2011-09-18T02:25:19	True	False	6	2018-06-15T20:08:49	2018-06-16T21:09:32	0.1559	6.6	
14															
15	397	371	11	1293	107103	False	2012-01-31T04:31:49	False	False	12	2018-06-15T12:24:52	2018-06-17T16:50:49	0.8345	45.99	
16	5970	2000	184	4856	13900	False	2009-04-08T12:46:57	False	False	6	2018-06-15T14:22:02	2018-06-17T07:25:43	0.335	4.14	
17	539	2226	15	57079	31875	False	2013-08-03T01:27:43	False	False	6	2018-06-14T03:59:07	2018-06-16T13:49:53	4.1299	17.83	
18	192	266	3	1555	5753	False	2009-08-12T12:22:57	True	False	12	2018-06-14T13:58:16	2018-06-16T18:42:48	1.3654	1.78	
19	390	550	14	730	22393	False	2009-05-27T15:57:58	False	False	11	2018-06-15T13:42:43	2018-06-17T15:42:16	1.4103	6.77	

Deux membres du groupe ont labellisé ensemble, ligne par ligne, environ 200 profils. Une astuce a été de filtrer et trier pour voir en priorité les profils qui correspondaient à des critères suspects et essayer de trancher les cas ambigus. Certains profils apparaissent assez rapidement comme humains ou comme suspects, ceux-ci ont été rapidement labellisés. Nous avons ensuite demandé à un LLM de confirmer ou infirmer nos raisonnements (qu'il a validés sauf pour deux cas que nous avons tranchés), et avons conjointement classé 200 profils supplémentaires, par paquets

de 50 où nous avons inversé les rôles : cette fois-ci, nous étions assignés à la deuxième lecture. Au total, notre échantillon final s'élève à 420 profils labellisés (après nettoyage), ce qui semble être un volume correct pour la mise en place de notre algorithme supervisé. Évidemment, un tel algorithme est plus performant sur de grandes quantités d'observations, mais il n'est pas humainement imaginable de labelliser les 1,8 million de profils, et nous avons souhaité éviter toute sorte d'interpolation pour ne pas induire de biais.

5. Analyse exploratoire des données (EDA)

Avant d'entamer la phase de modélisation, nous avons réalisé une analyse exploratoire sur notre échantillon final de 420 profils labellisés. Cette étape permet de valider nos hypothèses sur les caractéristiques discriminantes entre bots et humains, et d'orienter nos choix d'ingénierie des features.

5.1. Distribution des classes

Après nettoyage des valeurs nulles, notre base annotée compte **420 profils**, répartis de la manière suivante :

- **348 profils labellisés « Humain »** (env. 83 %)
- **72 profils labellisés « Bot »** (env. 17 %)

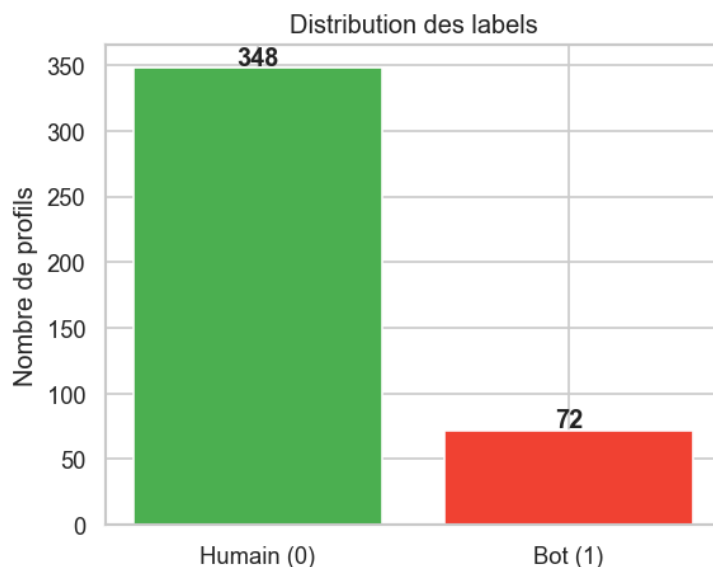


Figure 2: Distribution des labels dans l'échantillon annoté

Ce net déséquilibre naturel en faveur des comptes humains est classique sur les réseaux sociaux. Il implique qu'il faudra redoubler de vigilance lors de l'entraînement de nos modèles supervisés (stratification, pondération des classes) et dans le choix de nos métriques d'évaluation (F1-score, Precision-Recall AUC plutôt que la simple accuracy).

5.2. Analyse bivariée : Graphe social et Engagement

L'étude des distributions bivariées met en évidence des comportements distincts. Par exemple, si l'on observe la relation entre le nombre d'abonnés et le nombre d'abonnements (en échelle logarithmique), on remarque que les bots ont tendance à suivre beaucoup de comptes pour un nombre d'abonnés nettement inférieur. À l'inverse, les humains suivent une tendance plus linéaire et proportionnelle.

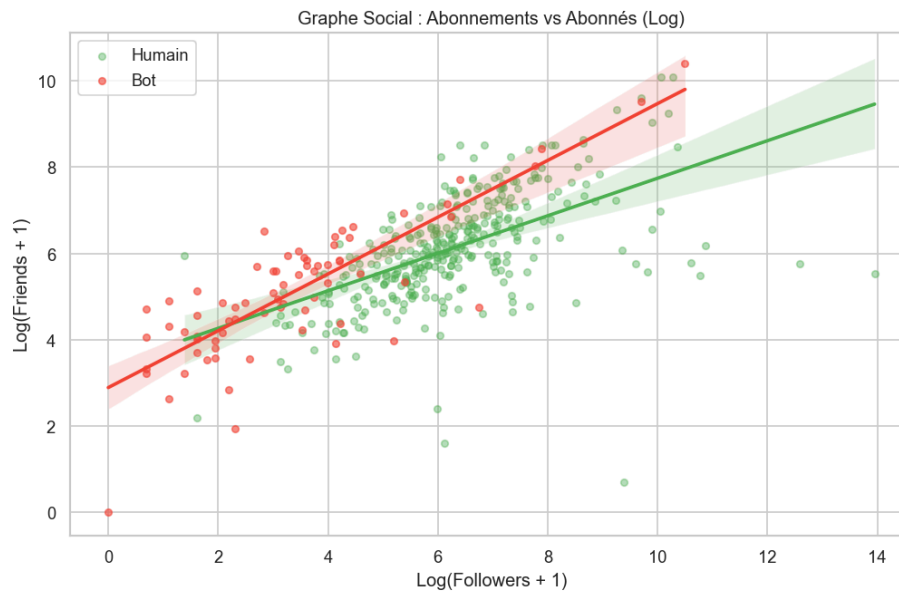


Figure 3: Graphe Social : Abonnements vs Abonnés (Log)

5.3. Personnalisation du profil

L'absence de personnalisation est un signal fort d'automatisation. Nos données le confirment de manière frappante : plus de 90 % des comptes humains de l'échantillon disposent d'une description (bio) complétée, tandis que près de la moitié des bots se contentent d'un profil vide à ce niveau.

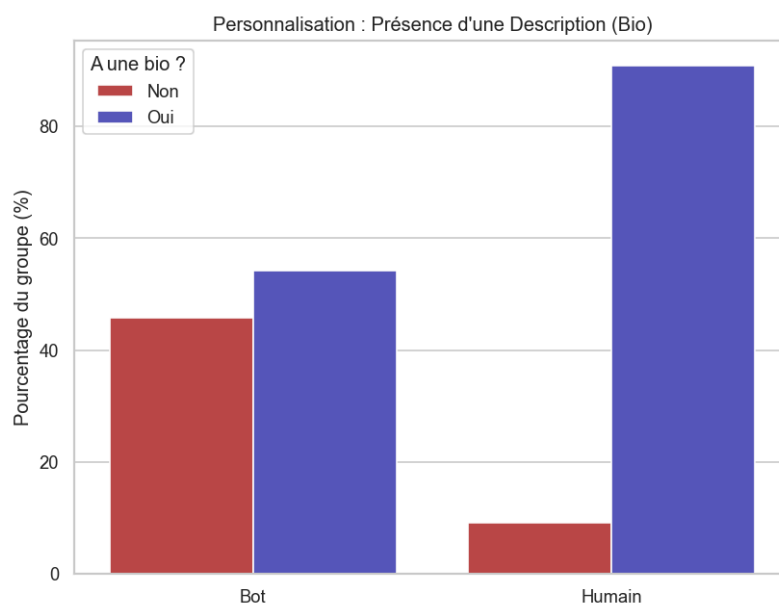


Figure 4: Présence d'une description selon la classe

5.4. Matrice de corrélation

L'analyse de la matrice de corrélation sur notre échantillon met en évidence plusieurs associations linéaires intéressantes qui viennent conforter nos hypothèses :

- **Influence et notoriété** : On observe une forte corrélation positive croisée entre le nombre d'abonnés (`followers_count`), d'apparitions dans des listes (`listed_count`) et le statut vérifié (`verified`). Les comptes suivis sont logiquement plus souvent répertoriés et certifiés.
- **Engagement** : Le volume de publications (`statuses_count`) est corrélé positivement au nombre de favoris émis (`favourites_count`). Les profils actifs dans la création de contenu le sont également dans l'interaction.
- **Ancienneté et personnalisation** : L'âge du compte (`account_age_days`) est corrélé négativement avec le maintien d'un profil par défaut (`default_profile`) : les vieux comptes ont presque tous fini par modifier leur profil au fil du temps. On constate aussi que l'absence de description dans la bio a tendance à coïncider avec un plus petit ratio friends/followers.
- **Cible (label)** : Bien qu'il s'agisse de corrélations point à point qui ne capturent pas les relations non-linéaires plus complexes, le label "bot" est positivement lié au ratio `followers_friends_ratio` (suivi massif sans retour) et à la densité de publications (`tweets_per_day_since_creation`), ce qui valide pleinement l'inclusion de ces métriques clés.

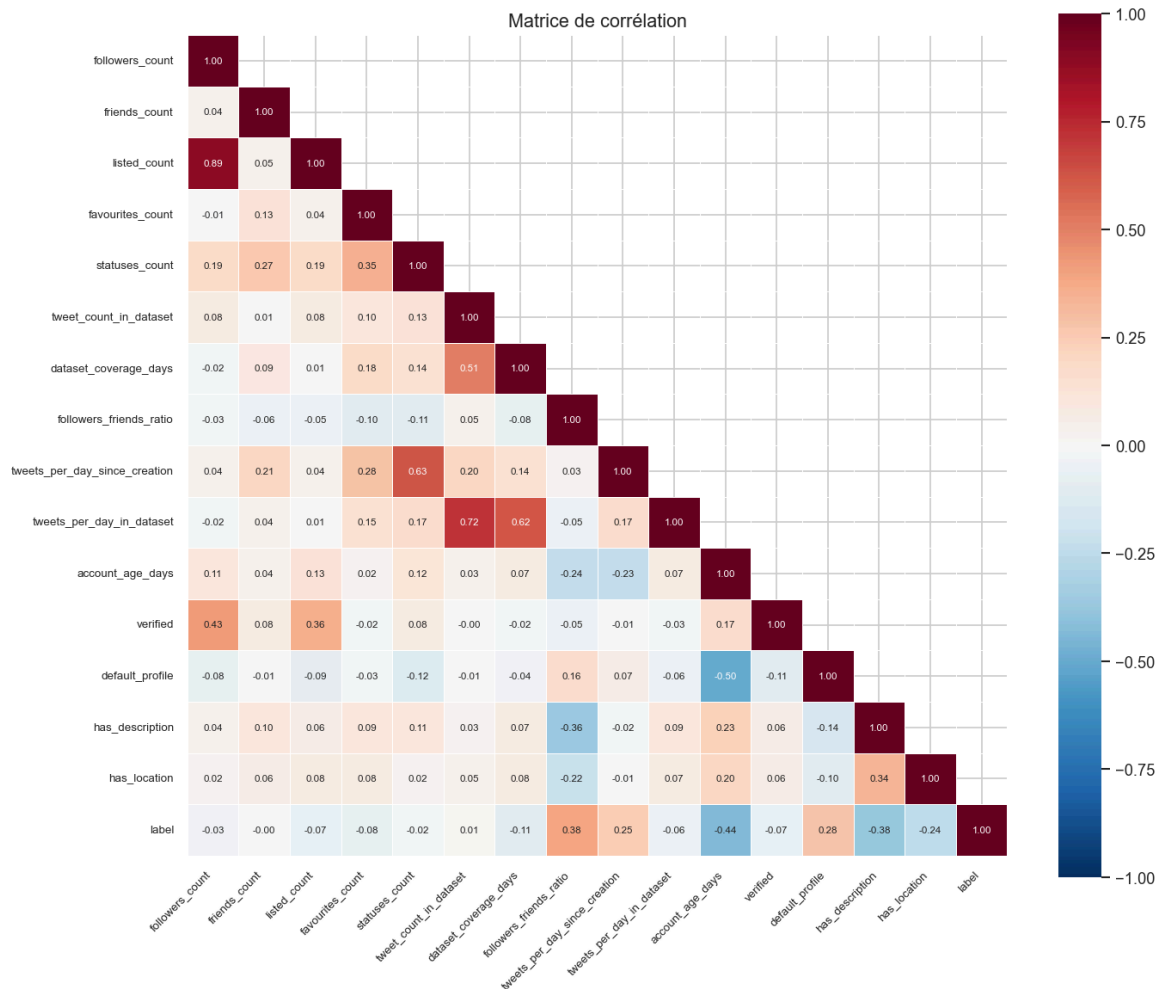


Figure 5: Matrice de corrélation des features

5.5. Synthèse dimensionnelle (Scores SPOT)

Nous avons également examiné la pertinence des dimensions extraites avec l'approche SPOT (Agressivité, Visibilité, Danger). Les boîtes à moustaches confirment la pertinence de certains de ces indicateurs :

- **L'agressivité** (fréquence combinée de tweets et d'ajouts d'amis) présente nettement plus de valeurs extrêmes chez les bots.
- **Le danger** (proportion de tweets contenant des URLs) est, en médiane, assez peu discriminant ici, mais on observe un étalement important des valeurs extrêmes chez les deux groupes.

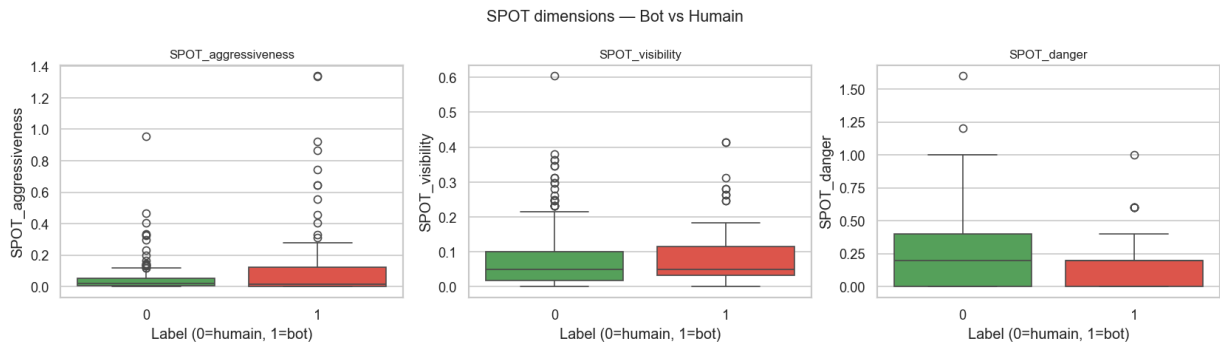


Figure 6: Dimensions SPOT évaluées sur notre échantillon : Agressivité, Visibilité, Danger

Ces constats exploratoires nous confortent dans le choix de ces métriques comme variables d'entrée pour nos algorithmes de modélisation.

6. Méthodologie de modélisation

6.1. Feature engineering

Nous avons commencé par calculer toutes les métriques utiles dans une `materialized view` en base de données, notamment celles repérées pour l'annotation : ratio friends/followers, âge du compte, fréquences de publication, agressivité SPOT, etc. À celles-ci s'ajoute une analyse des sources : le payload de l'API twitter donne un champ `source` qui est un lien html (tag `<a>`) duquel on peut extraire un nom. Nous avons extrait cette information avec la requête suivante :

```
SELECT
    COALESCE(
        NULLIF(trim(substring(raw_data->>'source' from '<a[^>]*>([^\<]*)</a>'), ' '),
        raw_data->>'source'
    ) AS tweet_source,
    COUNT(*) AS tweet_count
FROM
    tweets
GROUP BY
    1
ORDER BY
    2 DESC
;
```

À partir de là, on obtient une liste de plusieurs centaines de valeurs :

tweet_source	tweet_count
Twitter for Android	1879304
Twitter for iPhone	1660451
Twitter Web Client	460123
Twitter Lite	147823
TweetDeck	72768
Twitter for iPad	60325
Mobile Web (M2)	18292
IFTTT	13063
Facebook	12681
Paper.li	12206
Tweetbot for iOS	10523
Hootsuite Inc.	9002
FCBarcelonaBot	6857
...	

Nous avons donné cette liste à un LLM (Moonshot AI Kimi K2.6) afin qu'il identifie et cherche les sources connues comme étant des outils communs d'automatisation ou des bots. Cette approche est approximative : les bots plus sophistiqués peuvent parfois tenter de maquiller leur origine. En revanche, les bots étant constants dans leurs sources, nous avons également utilisé la variété des sources comme feature pour essayer de déterminer les bots, car un humain, même s'il utilise principalement une source, a souvent l'occasion d'en utiliser d'autres s'il a plusieurs appareils (en général : Client Web, iOS, Android, iPadOS...)

Afin d'entraîner nos modèles, nous avons structuré nos attributs (**features**) en trois grandes catégories, en nous inspirant de la littérature existante sur la détection d'anomalies sur les réseaux sociaux.

- **Caractéristiques liées au profil (User-based)** : Nous prenons en compte la présence d'éléments descriptifs (`has_description`, `has_url`, `has_location`, `description_length`), la structure du pseudonyme (`screen_name_length`, `screen_name_has_digits`), ainsi que des mesures d'influence et de socialisation telles que la réputation ($\text{ratio } \frac{\text{followers}}{\text{followers} + \text{friends} + 1}$) ou le ciblage des listes (`listed_per_follower`). Le ratio **followers/friends** est une métrique classique pour identifier les bots qui suivent massivement d'autres comptes sans être suivis en retour (Chu et al., 2012). L'ancienneté du compte (`account_age_days`) est également pertinente, les vagues de création massive de faux comptes étant fréquentes.
- **Caractéristiques d'activité (Activity-based)** : L'intensité de publication est évaluée via le volume total de tweets (`tweet_count`), la fréquence historique depuis la création (`tweet_frequency`), ou encore la densité d'activité calculée pendant la période de collecte (`tweets_per_day_in_dataset`). Ces mesures traduisent le comportement intensif caractéristique de nombreux comptes automatisés. L'intérêt pour les interactions est évalué à travers le taux de favoris donnés (`favourites_per_status`). Le délai depuis le dernier tweet (`days_since_last_tweet`) ou l'étendue de l'observation (`observation_span_days`) donnent une mesure de l'assiduité du compte.
- **Caractéristiques de contenu (Content-based)** : Ces attributs caractérisent la nature des messages émis. Nous calculons la fréquence d'utilisation d'URLs, de hashtags, et de mentions (`urls_per_tweet`, `hashtags_per_tweet`, `mentions_per_tweet`), ces derniers étant souvent saturés par les bots de spam à visée promotionnelle. Enfin, l'orientation vers le relais d'information (au lieu de la production originale) est mesurée par la proportion de retweets (`retweet_rate`) et le taux de réponses (`reply_rate`). Nous y ajoutons bien sûr le taux de sources suspectes ou automatisées (`bot_source_ratio`) et la diversité des outils employés (`unique_sources`) évoqués plus haut.

Le script SQL complet disponible sur GitHub.

6.2. Préparation des données pour l'apprentissage

Avant d'injecter nos données dans les modèles, plusieurs transformations ont été appliquées pour garantir leur exploitabilité.

6.2.1. Nettoyage et exclusions

Nous avons supprimé du jeu d'entraînement les identifiants explicites (`user_id`, `screen_name`, `name`), ainsi que les URL de profil et descriptions sous format texte, le modèle n'étant conçu que pour traiter des données numériques. Les dates de création et d'activité (`created_at`, `last_tweet_at`, etc.) ont également été écartées, l'essentiel de l'information temporelle étant déjà capturée et structurée sous forme de features (âge du compte, jours depuis le dernier tweet). Enfin, les valeurs booléennes (`has_location`, `has_url`...) ont été converties au format binaire entier, et les valeurs nulles (souvent causées par une absence de tweets permettant de calculer les ratios) ont été remplacées par 0.

6.2.2. Traitement de la colinéarité

Une caractéristique frappante de l'ingénierie des caractéristiques est l'apparition de forte redondance entre certains champs. Notre matrice de corrélation exploratoire l'avait révélé : des indicateurs tels que `followers_count` et `listed_count` peuvent être extrêmement liés. Bien qu'un modèle à base d'arbres puisse survivre à cette redondance (il sélectionnera les meilleures partitions de façon stochastique), une forte colinéarité nuit à l'interprétabilité globale (cf. *feature importance*) et alourdit l'entraînement. Malgré cela, il n'y a aucune corrélation qui soit supérieure à 90%

6.2.3. Choix du type de normalisation

L'approche de mise à l'échelle (**scaling**) n'est pas universelle. Dans ce projet impliquant une comparaison entre deux familles d'algorithmes fondamentalement différentes :

- Pour le modèle de **Forêts Aléatoires (Random Forest)**, aucune standardisation (`StandardScaler`) n'a été appliquée. Les algorithmes d'arbres partitionnent l'espace de manière orthogonale en divisant les fonctionnalités sur des seuils de décision (ex: $X > 100$). Une transformation monotone ou affine ne change pas la structure des nœuds ; elle est complètement invariante à l'échelle.
- À l'inverse, l'approche par **Machine à Vecteurs de Support (SVM)**, explorée par la suite, ainsi que le **Clustering**, reposent formellement sur le calcul matriciel de distances euclidiennes. Sur ces derniers, un espace de variables déséquilibré fausse l'hyperplan optimal. Il requiert un processus de standardisation systématique des caractéristiques, voire parfois une transformation logarithmique (**log1p**) sur les données comportant de grandes disproportions et d'importantes asymétries de distribution statistiques (**heavy-tailed data**).

6.3. Modélisation Supervisée : Random Forest

Pour expérimenter la classification supervisée, nous avons retenu l'algorithme **Random Forest** (Forêts Aléatoires). Ce modèle ensembliste repose sur la construction d'un grand nombre d'arbres de décision appliqués sur des sous-échantillons aléatoires. Il est naturellement robuste face au sur-apprentissage et accommode sans heurt un ensemble hétéroclite de features.

6.3.1. Échantillonnage et validation

Compte tenu du net déséquilibre de nos classes (17% de bots pour 83% d'humains), nous devons éviter d'avoir par hasard une représentation nulle des bots dans nos ensembles de validation. Le jeu de données a été scindé à l'aide de la fonction `train_test_split` avec les proportions 80% (Entraînement - `X_train`) et 20% (Test - `X_test`), en spécifiant un argument de **stratification**. Cette précaution garantit mathématiquement que la distribution initiale des classes est honorée dans chacun des échantillons.

6.3.2. Optimisation des hyperparamètres

Afin d'ajuster finement les performances du Random Forest, nous avons mis en place une exploration des hyperparamètres via une recherche sur grille exhaustive (`GridSearchCV`).

Les paramètres de cette grille comprenaient notamment :

- Le nombre d'arbres (`n_estimators`).
- L'architecture de la profondeur (`max_depth`).
- La flexibilité fine des feuillages terminaux (`min_samples_split`, `min_samples_leaf`).
- La pondération de classes (`class_weight = "balanced"`) face à l'asymétrie Humains/Bots.

L'optimisation a été pilotée en validation croisée stratifiée sur le sous-ensemble d'entraînement à 5 passes (`StratifiedKFold`, `n_splits=5`), employant, non pas la précision (**Accuracy**), mais spécifiquement le **F1-score** comme unique boussole d'apprentissage.

6.3.3. Importance des features

Un avantage pratique du Random Forest est qu'il donne une forme d'explicabilité assez directe via `feature_importances_`. L'idée n'est pas de dire qu'une variable "cause" le label bot, mais de mesurer quelles variables ont le plus souvent permis aux arbres de faire des séparations utiles.

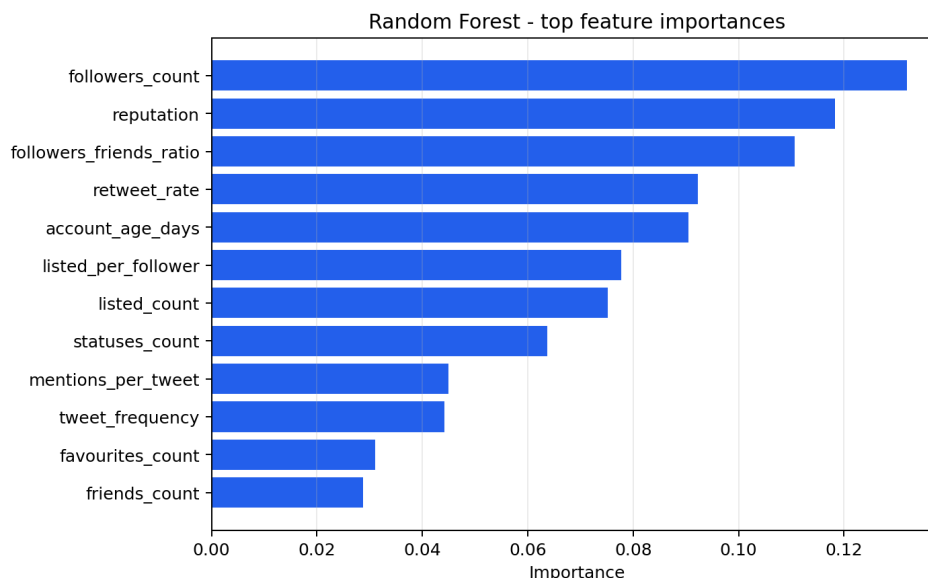


Figure 7: Features les plus importantes dans le modèle Random Forest

Dans notre modèle, les variables qui ressortent le plus sont `followers_count`, `reputation` et `followers_friends_ratio`. Ce sont toutes des variables liées à la structure sociale du compte : combien il est suivi, combien il suit d'autres comptes, et l'équilibre entre les deux. Ensuite viennent `retweet_rate` et `account_age_days`, qui capturent davantage le comportement : part de retweets et ancienneté du compte.

C'est aussi pour cette raison que nous n'avons pas utilisé de PCA pour expliquer le Random Forest. Une PCA peut être utile pour visualiser les profils en deux dimensions, mais elle mélange les variables entre elles. Ici, garder les features originales permet de dire plus simplement quels signaux le modèle utilise réellement.

6.3.4. Évaluation et Matrice de Confusion

L'évaluation finale de notre modèle s'est faite sur l'ensemble de Test (20% des données), laissé jusque-là parfaitement invisible à l'apprentissage. Afin d'appréhender toute la granularité de ces résultats, particulièrement dans un contexte déséquilibré, nous nous sommes appuyés sur la matrice de confusion usuelle ainsi que sur plusieurs métriques, définies mathématiquement par les taux de Vrais Positifs (V_p), Faux Positifs (F_p), Vrais Négatifs (V_n) et Faux Négatifs (F_n) :

- **L'exactitude (Accuracy)** : ratio de prédictions correctes globales : $\frac{V_p + V_n}{\text{Total}}$. Bien qu'élevée à 0.917 (soit 91.7% de bonnes intuitions globales), elle peut s'avérer trompeuse en asymétrie de classes (il "suffit" d'étiqueter systématiquement la classe majoritaire pour obtenir un score honorable).
- **La précision (Precision)** : $\frac{V_p}{V_p + F_p}$. Parmi l'ensemble des comptes signalés comme "Bots" par notre modèle, quelle est la proportion réelle de vrais bots ? Elle s'élève ici à 0.684.
- **Le rappel (Recall ou Sensibilité)** : $\frac{V_p}{V_p + F_n}$. Parmi tous les bots **existant réellement** dans l'échantillon de test, combien le modèle a-t-il pu en identifier ? Avec notre configuration, cette métrique culmine à 0.929 (près de 93% des bots sont débusqués).
- **Le F1-Score** : la moyenne harmonique de la précision et du rappel, calculée selon $2 \cdot \frac{P \cdot R}{P + R}$. C'est le juge de paix. Face au déséquilibre de classe, notre modèle a abouti à un très bon score certifié par validation croisée de 0.788.

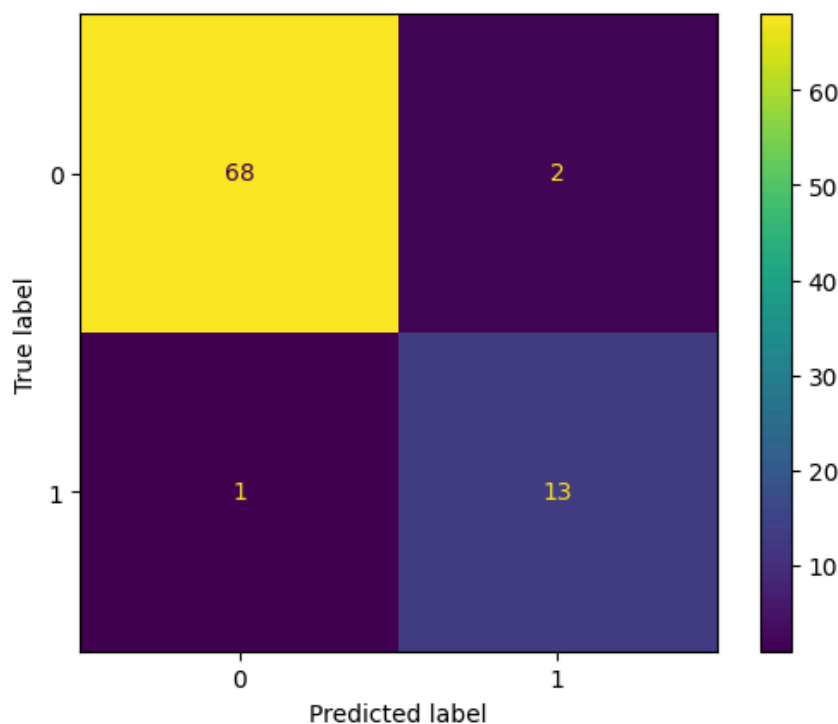


Figure 8: Matrice de confusion du modèle Random Forest sur l'ensemble de Test

Cette matrice, et en particulier l'excellent score de rappel, illustre le compromis de "paranoïa utile" atteint par le modèle lors de la recherche des hyperparamètres (le poids **balanced** étant retenu). Il s'avère doté d'une redoutable capacité de détection des profils bots isolés, ne laissant s'échapper qu'une infime proportion d'entre eux, acceptant paradoxalement d'inclure quelques fausses alertes humaines en collatéral.

Si l'on se place dans le business case typique d'une plateforme de réseaux sociaux qui cherche à éliminer le trafic lié aux bots, il est préférable d'avoir recall élevé quitte à avoir plus de faux positifs : l'humain faux positif peut prouver qu'il n'est pas un robot (captcha, code par e-mail, action manuelle), mais un robot qui passe entre les mailles du filet ne sera jamais débusqué.

6.4. Modélisation Supervisée : SVM

En plus du Random Forest, et parce qu'il nous semblait intéressant de mettre en pratique ce que nous avons vu en cours, nous avons aussi décidé d'implémenter un **modèle de machine à vecteur de support (SVM)**. Pour rappel, le SVM est un modèle qui classe les données en trouvant une ligne optimale ou un hyperplan (dans un cas non linéairement séparable) permettant de maximiser la distance entre chaque classe dans un espace.

6.4.1. Échantillonnage et validation

Notre SVM a été séparé, comme pour le Random Forest, grâce à la fonction `train_test_split` avec 80% des données qui ont servi en entraînement (un échantillon de 336 données - `x_train`) et 20% des données qui ont servi en tant que données de test (soit un échantillon de 84 samples - `x_test`). Comme dis dans la partie portant sur le Random Forest, étant donné le déséquilibre présent entre nos deux classes (Bot - Non-bot), nous avons dû spécifier un argument de **stratification**.

6.4.2. Réduction dimensionnelle

6.4.2.1. Réduction par ACP

Contrairement au Random Forest, le SVM est sensible à l'échelle et à la corrélation des features. Nous avons donc appliqué une **Analyse en Composantes Principales (ACP)** après standardisation (`StandardScaler`), dans le double objectif de décorrélérer les variables et de réduire le bruit.

La sélection du nombre de composantes à conserver a été faite automatiquement en fixant un **seuil de variance expliquée cumulée à 95%**. Sur les 31 composantes possibles, **23 suffisent à capturer 95% de l'information**, comme le montre le graphique ci-dessous. Les 8 composantes restantes ne portent que du bruit marginal et auraient risqué de dégrader les performances du SVM.

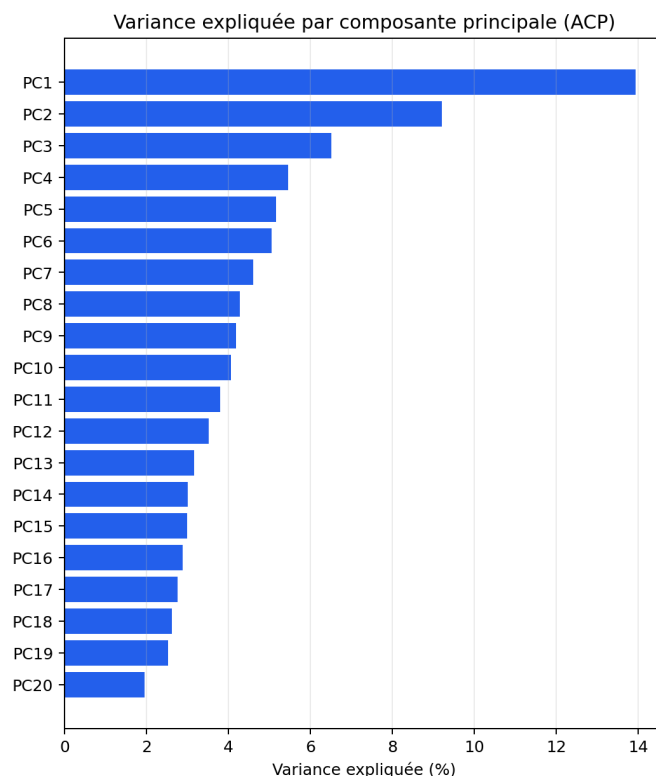


Figure 9: Variance expliquée par composante principale — 23 composantes retenues sur 31 au seuil de 95%

6.4.3. Optimisation des hyperparamètres

Lors de notre analyse des données, nous avons eu une première intuition qui était que notre cas d'étude ne serait pas linéairement séparable, donc que par défaut nous n'utiliserions pas de **noyau linéaire**.

Dans l'objectif de trouver les meilleurs hyperparamètres, nous avons mis en place une exploration des hyperparamètres via une recherche sur grille exhaustive (**GridSeachCV**).

Les paramètres de cette grille comprenaient notamment :

- Le type de noyau (**kernel**).
- Le coefficient de régularisation inversé (**C**).

Par défaut, la pondération de classes (**class_weight**), dû à l'asymétrie Humains/Bots, a été faite avec l'hyperparamètre **balanced**.

Pour rester sur les mêmes base que pour le Random Forest afin que les comparaison soit cohérente, l'optimisation a été faite de la même manière. L'unique boussole d'apprentissage a été cette fois aussi non pas la précision (**Accuracy**), mais le **F1-score**.

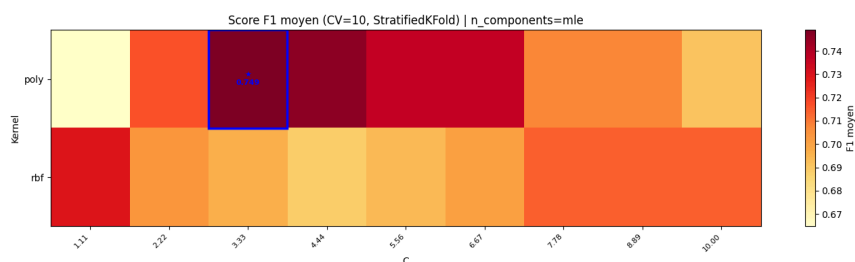


Figure 10: Résultat du GridSeachCV

Pour le cas de notre SVM, on remarque que les meilleurs paramètres trouvés par le GridSearchCV sont :

- Le noyau `rbf`.
- Un coefficient de régularisation inversé égale à **0.747**.

Notre intuition lors de l'analyse des données a été bonne, et ça nous donne un score F1 maximisé à 0.727.

6.4.4. Importance des features

Contrairement au Random Forest, le SVM avec noyau RBF n'expose pas directement d'importance des features. La décision repose sur des distances dans un espace de haute dimension transformé par la PCA (31 $\rightarrow$ 23 composantes), ce qui rend l'interprétation par variable moins immédiate.

On peut toutefois examiner les loadings des composantes principales pour identifier quelles features structurent l'espace dans lequel le SVM trace son hyperplan. La composante dominante (PC1, 13,9 % de variance) est portée par `description_length`, `retweet_rate`, `has_description` et `account_age_days`, des signaux de personnalisation et de comportement. La PC2 (9,2 %) capture la densité d'activité dans le corpus (`tweet_count`, `tweets_per_day_in_dataset`). La PC4 (5,5 %) est quasi exclusivement portée par `bot_source_ratio` (loading 0,74), ce qui signale une dimension très discriminante liée aux sources automatisées.

Ces axes rejoignent les features identifiées comme importantes par le Random Forest (activité, graphe social, comportement), même si le chemin pour y arriver est moins direct.

6.4.5. Évaluation

6.4.5.1. Matrice de confusion

Tout comme pour le Random Forest, l'évaluation finale de notre SVM s'est faite sur l'ensemble de Test (20% des données), laissé jusque-là parfaitement invisible à l'apprentissage. Afin d'appréhender toute la granularité de ces résultats, particulièrement dans un contexte déséquilibré, nous nous sommes appuyés sur la matrice de confusion usuelle ainsi que sur plusieurs métriques, définies mathématiquement par les taux de Vrais Positifs (V_p), Faux Positifs (F_p), Vrais Négatifs (V_n) et Faux Négatifs (F_n) :

- **L'exactitude (Accuracy)** : ratio de prédictions correctes globales : $\frac{V_p + V_n}{\text{Total}}$. Elle s'élève à 0.881 (soit 88.1% de bonnes prédictions globales). Comme pour le Random Forest, elle peut s'avérer trompeuse en asymétrie de classes.
- **La précision (Precision)** : $\frac{V_p}{V_p + F_p}$. Parmi l'ensemble des comptes signalés comme "Bots" par notre modèle, quelle est la proportion réelle de vrais bots ? Elle s'élève ici à 0.625.
- **Le rappel (Recall ou Sensibilité)** : $\frac{V_p}{V_p + F_n}$. Parmi tous les bots **existant réellement** dans l'échantillon de test, combien le modèle a-t-il pu en identifier ? Avec notre configuration, cette métrique atteint 0.714 (10 bots détectés sur 14).
- **Le F1-Score** : la moyenne harmonique de la précision et du rappel, calculée selon $2 \cdot \frac{P \cdot R}{P + R}$. C'est le juge de paix. Face au déséquilibre de classe, notre modèle a abouti à un score certifié par validation croisée de 0.727.

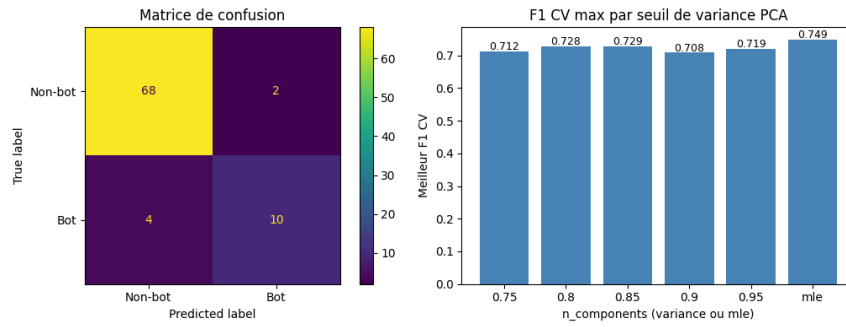


Figure 11: Matrice de confusion du modèle SVM sur l'ensemble de Test

Cette matrice illustre un comportement plus équilibré que le Random Forest : le SVM détecte 10 bots sur 14 (recall de 0,71) tout en générant seulement 6 faux positifs sur 70 non-bots (precision de 0,62). Le compromis est donc moins agressif que le RF — qui privilégiait un recall très élevé au prix de davantage de fausses alertes.

Ce comportement plus prudent peut s'expliquer par la nature même du SVM : il cherche à maximiser la marge entre les classes dans l'espace PCA, ce qui tend à produire une frontière de décision plus conservative. Dans notre cas d'usage (détection de bots), un recall de 0,71 reste acceptable, mais le RF reste préférable si l'on priorise la détection exhaustive.

6.4.5.2. ROC-AUC

La courbe ROC (Receiver Operating Characteristic) complète l'analyse de la matrice de confusion en évaluant les performances du modèle sur **tous les seuils de décision possibles**, et non plus uniquement au seuil par défaut $f = 0$. Pour chaque seuil, elle trace le taux de vrais positifs (recall) en fonction du taux de faux positifs ($FPR = \frac{F_p}{F_p + V_n}$). L'aire sous cette courbe, l'**AUC** (Area Under the Curve), résume cette performance en un unique scalaire : 1,0 représente un classifieur parfait, 0,5 un classifieur aléatoire.

Notre SVM obtient une **AUC de 0,933**, ce qui signifie que dans 93,3 % des cas, il attribue un score de décision plus élevé à un vrai bot qu'à un vrai humain. C'est un indicateur de bonne **discrimination globale**, indépendamment du seuil choisi.

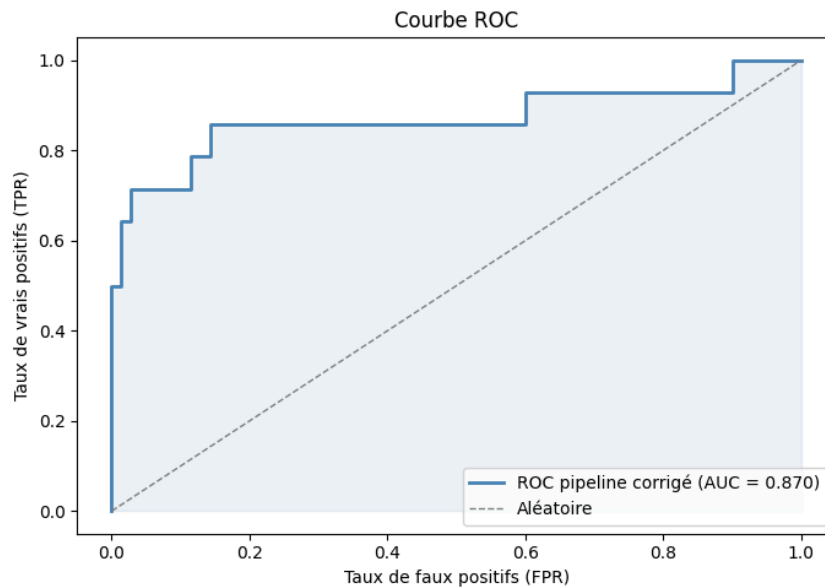


Figure 12: Courbe ROC du modèle SVM — AUC = 0,933

Cette AUC élevée nuance le F1 de 0,667 observé au seuil par défaut : le modèle **sait** bien ordonner les profils par risque, mais son point de fonctionnement naturel n'est pas optimal pour notre use case. En abaissant le seuil de décision, on pourrait augmenter le recall au-delà de 0,71 au prix d'une précision réduite.

6.5. Conclusion

6.5.1. interprétation des résultats

Les deux modèles supervisés ont été entraînés et évalués dans les mêmes conditions : 420 profils labellisés, split 80/20 stratifié, validation croisée à 5 plis, et `class_weight=balanced` pour tenir compte du déséquilibre de classes (17 % de bots).

Le **Random Forest s'impose** comme le meilleur classifieur dans notre contexte : F1 de 0,788 contre 0,667 pour le SVM, et surtout un **recall** de 0,929, ce qui signifie qu'il ne laisse passer que 1 bot sur 14. C'est l'avantage clef pour notre use case de détection de bots, où un faux négatif (un bot non détecté) est plus problématique qu'un faux positif (un humain signalé à tort). Le SVM, lui, produit une frontière de décision plus conservative dans l'espace PCA, ce qui explique son **recall** plus faible (0,714). Son **AUC de 0,933** montre qu'il ordonne bien les profils par risque, mais son seuil de décision par défaut n'est pas optimal pour notre objectif.

6.5.2. Point de vigilance

Cela dit, les deux modèles partagent une limite **commune** et **fondamentale** : la **qualité des labels** sur lesquels ils ont été entraînés. La labellisation des 420 profils a été réalisée **manuellement**, en s'appuyant sur certaines heuristiques (ratio de retweets, sources automatisées, fréquence de tweets, personnalisation du profil, fréquence des tweets, ...). Cette démarche est inévitablement subjective : deux annotateurs pourraient classer différemment un profil actif mais atypique. Certains bots discrets pu être étiquetés humains, et inversement. Dans ce contexte, les scores obtenus reflètent autant la cohérence du modèle avec nos propres critères d'annotation que sa capacité à détecter des bots réels. Un modèle avec un **recall** de 0,93 peut simplement avoir appris à reproduire fidèlement nos biais d'annotation.

6.5.3. Pour conclure

En résumé, le **Random Forest** est l'approche à privilégier pour la détection de bots, mais ses performances doivent être interprétées avec prudence : elles sont plafonnées par la qualité et la subjectivité de notre ground truth.

7. Modélisation Non-Supervisée

L'objectif du non-supervisé était différent de celui du Random Forest. Ici, on ne cherche pas seulement à prédire les 420 labels déjà connus, mais surtout à voir si la structure globale des profils Twitter fait apparaître des groupes naturels. Nous avons donc testé deux approches : MiniBatch K-means à grande échelle, puis un clustering spectral sur l'échantillon annoté.

7.1. MiniBatch K-means sur l'ensemble des profils

Pour K-means, nous avons voulu exploiter le fait que nous avons déjà une base de features pour tous les utilisateurs. Un K-means classique n'est pas pratique sur environ 1,8 million de profils, surtout avec une machine personnelle. Nous avons donc utilisé `MiniBatchKMeans`, qui met à jour les centroïdes par petits lots et permet de travailler en streaming.

Le pipeline était le suivant : les 420 profils labellisés ont été exclus de l'entraînement, puis les autres profils ont été lus depuis PostgreSQL par lots de 50 000. Une première passe a servi à ajuster un `StandardScaler` avec `partial_fit`, puis une deuxième passe a entraîné le MiniBatch K-means, toujours par batch. Cela évite de charger toute la matrice en mémoire et évite aussi une fuite de données entre les profils labellisés et l'entraînement non-supervisé.

Au final, le modèle a été entraîné sur 1 820 857 profils, avec $K=2$. Les 420 profils labellisés ont ensuite été projetés dans le même espace standardisé, puis affectés au cluster le plus proche.

Cluster	Profilis sonde	Bots	Taux de bots
0	265	11	4,2 %
1	155	61	39,4 %

Le résultat est intéressant mais pas suffisant comme classifieur direct. Le cluster 1 concentre beaucoup plus de bots que le cluster 0, mais il contient aussi 94 profils humains. Si on force une lecture simple où le cluster 1 devient le cluster suspect, on obtient une accuracy de 0,750, une precision de 0,394, un recall de 0,847 et un F1-score de 0,537. Le recall est donc correct, mais la precision reste faible : le clustering retrouve une zone de profils suspects, sans tracer une frontière propre entre humains et bots.

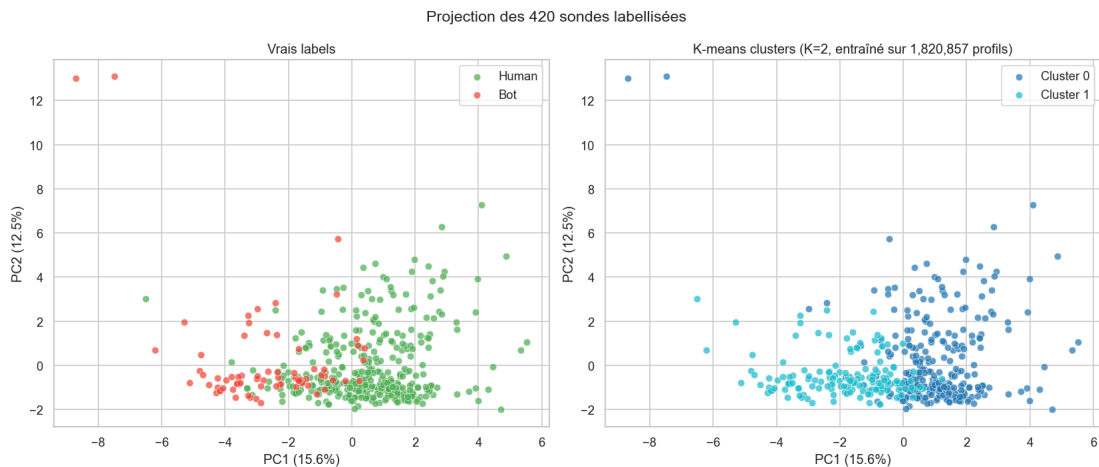


Figure 13: Projection PCA des 420 profils labellisés avec les clusters MiniBatch K-means entraînés sur les profils non labellisés

Une limite cependant est également la qualité de notre labélisation. Certains profils humains sont très actifs et peuvent se retrouver dans le cluster suspect, tandis que certains bots sont plus discrets et se retrouvent dans le cluster humain. Le clustering non-supervisé ne peut pas remplacer un modèle supervisé, mais il peut aider à identifier des zones de suspicion. De même, notre labélisation manuelle est **subjective** : elle est basée sur une analyse humaine de métriques d'activité, mais il y a une grande part d'interprétation et d'intuition. On peut mettre en doute le fait que le k-means s'éloigne de la réalité et penser que peut-être notre labélisation passe à côté de certaines nuances. Le clustering non-supervisé peut donc aussi servir à vérifier la pertinence de nos labels.

7.2. Clustering spectral sur les 420 profils

Nous avons aussi testé un clustering spectral sur les 420 profils annotés. L'idée était de ne pas se limiter à des centroïdes comme dans K-means. Le clustering spectral construit d'abord un graphe de similarité entre profils, ici avec les 10 plus proches voisins, puis travaille dans l'espace spectral associé au graphe. En pratique, la matrice des vecteurs propres du graphe peut faire ressortir des relations sous-jacentes qui ne sont pas bien séparées dans l'espace original.

Cette approche a été lancée avec un binaire Rust `spectral-rs` emprunté à l'application GraphXR sur laquelle certain d'entre nous ont travaillé, sur un graphe de 420 noeuds et 2 174 arêtes. L'algorithme a trouvé 3 clusters. Les scores par rapport aux labels manuels restent modestes : $ARI = 0,253$ et $NMI = 0,112$.

Cluster spectral	Profil	Bots	Taux de bots
0	11	1	9,1 %
1	63	32	50,8 %
2	346	39	11,3 %

Le cluster 1 est le plus suspect, avec environ la moitié de bots, mais il ne récupère que 32 bots sur 72. Si on le lit comme cluster bot, on obtient environ 0,831 d'accuracy, 0,508 de précision, 0,444 de recall et 0,474 de F1-score. Ce n'est donc pas meilleur que le Random Forest, mais cela donne une autre lecture : le spectral isole un petit groupe dense de profils atypiques plutôt qu'un groupe large de suspects.

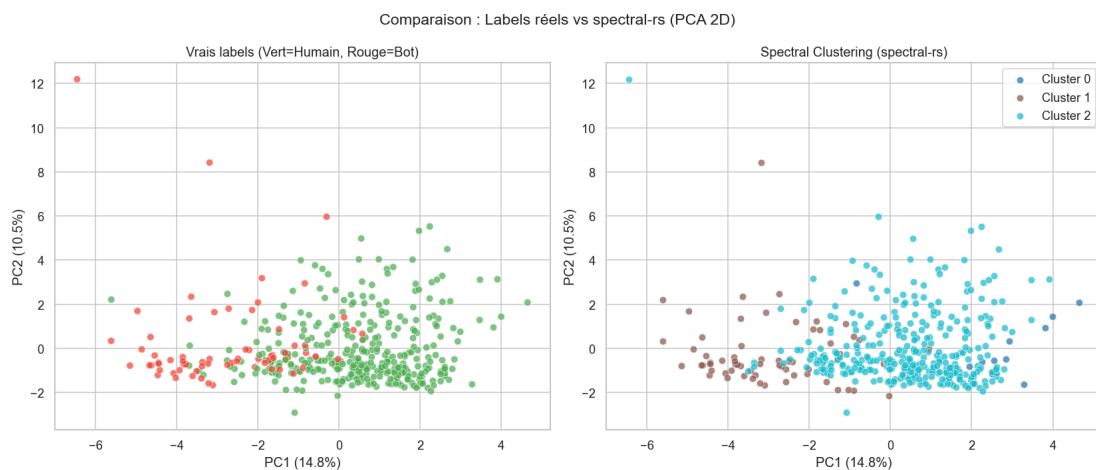


Figure 14: Projection PCA des 420 profils avec les clusters spectral-rs

8. Discussion, résultats et conclusion

8.1. Bilan des modèles

Les modèles donnent des résultats assez cohérents avec ce qu'on pouvait attendre. Le Random Forest est le plus efficace quand on veut prédire directement le label bot/humain, car il apprend la frontière à partir de nos annotations. Les méthodes non-supervisées, elles, sont plus exploratoires : elles font ressortir des zones de profils suspects, mais elles ne remplacent pas un modèle supervisé pour prendre une décision profil par profil.

Méthode	Accuracy	Precision	Recall	F1
Random Forest	0,917	0,684	0,929	0,788
SVM	0,881	0,625	0,714	0,667
MiniBatch K-means	0,750	0,394	0,847	0,537
Spectral clustering	0,831	0,508	0,444	0,474

Le Random Forest a surtout l'avantage d'un recall élevé : il retrouve 13 bots sur 14 dans le test set. C'est le comportement que l'on préfère dans notre cas d'usage, parce qu'un faux positif humain peut être révérifié, alors qu'un bot qui passe sous le radar reste actif. La précision plus faible veut simplement dire que le modèle est un peu agressif dans sa détection.

8.2. Random Forest contre clustering

La comparaison RF contre K-means n'est pas parfaitement symétrique. Le Random Forest est entraîné à reproduire nos labels, tandis que K-means ne connaît pas la notion de bot. Le fait que le cluster 1 du MiniBatch K-means contienne 39,4 % de bots reste donc un signal intéressant : il montre que les features de comportement regroupent naturellement une partie des profils suspects. En revanche, ce cluster contient encore beaucoup d'humains, donc il ne peut pas être utilisé seul comme règle de décision.

Le spectral clustering donne une lecture un peu différente. Il isole un groupe plus petit et plus dense, avec un taux de bots plus élevé, mais il manque beaucoup de bots. Cela peut être utile pour une analyse exploratoire ou pour repérer certains types de profils très marqués, mais pas pour une détection complète.

Au final, l'approche supervisée est plus opérationnelle. Le non-supervisé sert plutôt à comprendre la topographie des profils et à vérifier que nos features ne sont pas totalement arbitraires.

8.3. Limites

La limite principale reste la labellisation. Nous n'avons que 420 profils annotés, et même si le travail a été fait sérieusement, une part de subjectivité reste présente. Certains comptes peuvent être hybrides, automatisés partiellement, ou simplement très actifs pendant la Coupe du Monde.

Le contexte joue aussi beaucoup. Pendant un événement mondial, des humains peuvent retweeter massivement, utiliser beaucoup de hashtags, ou parler presque toujours du même sujet. Cela rapproche certains comportements humains de comportements que l'on associe habituellement aux bots. Nos résultats sont donc valables pour ce corpus et cette période, mais ils ne suffisent pas à généraliser à tout Twitter.

Enfin, nos features restent assez classiques : profil, activité, sources, URLs, hashtags, retweets. Elles capturent beaucoup de signaux utiles, mais elles ne comprennent pas réellement le contenu textuel des tweets.

8.4. Ouverture : POC sur la répétition sémantique

Une idée complémentaire que nous avons eu est celle d'exploiter la principale absente de nos données : le contenu textuel des tweets. Nous avons d'abord voulu tenter de calculer la distance de Levenshtein, disponible nativement dans PostgreSQL, entre les tweets d'un même utilisateur. Cependant le calcul est très coûteux (Postgres refuse même de le faire à partir d'un certain nombre d'entrées) et il ne capture pas la sémantique.

L'idée qui en découle est de calculer des embeddings vectoriels des tweets, et ensuite de mesurer la distance entre ces embeddings pour chaque utilisateur. Cela permettrait de quantifier la répétition sémantique : un bot répète souvent les mêmes idées ou reste dans un espace thématique étroit, tandis qu'un humain varie davantage. Cette addition en tant que composante principale de notre analyse est très intéressante, mais elle nécessite un temps de développement et de calcul important si on veut explorer les 4.5 millions de tweets. Il ne nous est donc pas possible de le faire sérieusement dans le cadre de ce projet.

Cependant, pour aller un peu plus loin, nous avons lancé un *Proof Of Concept* avec l'aide de *Codex* (harness et modèle d'OpenAI) séparé sur les 420 profils labellisés. L'idée était de créer des embeddings des tweets avec `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, puis de calculer des métriques de répétition sémantique par utilisateur.

Ce POC a été gardé à part dans `src/ml/semantic_poc/`, justement pour ne pas le mélanger avec le coeur du rapport. Sur le même split que le Random Forest, l'ajout de ces features améliore légèrement les scores : accuracy de 0,917 à 0,929, recall de 0,929 à 1,000, et F1 de 0,788 à 0,824.

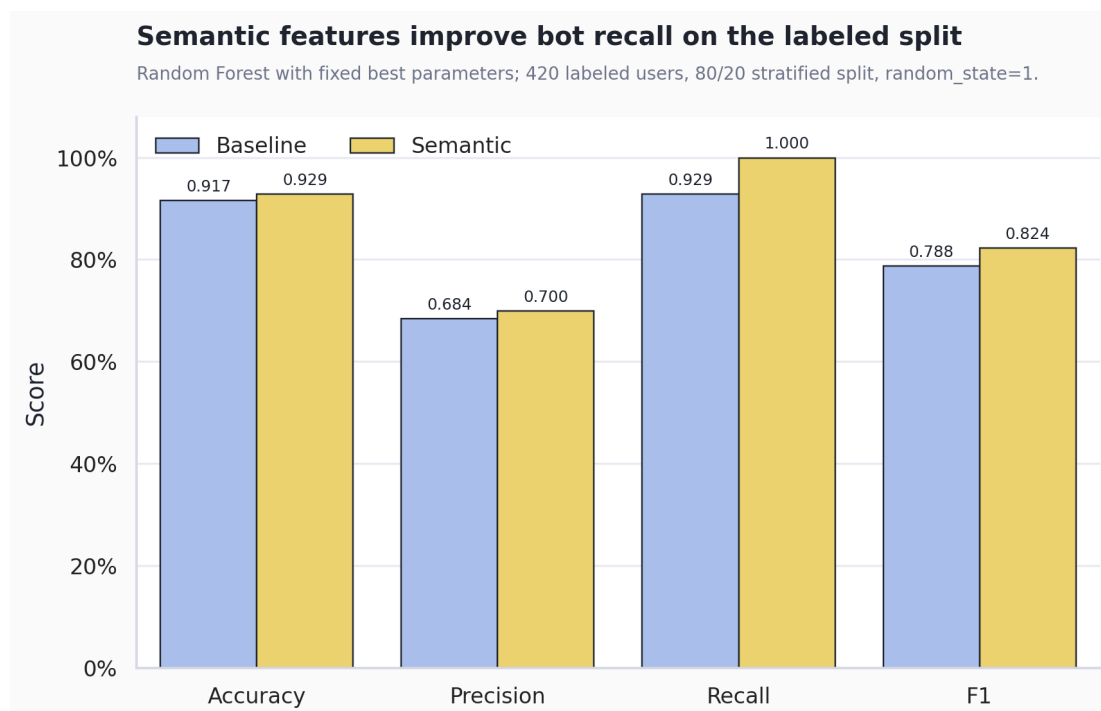


Figure 15: Comparaison Random Forest sans et avec les features sémantiques du POC

Les métriques sémantiques ne remplacent pas les features existantes mais ajoutent un signal complémentaire, avec des corrélations faibles ou modérées avec les features déjà présentes.

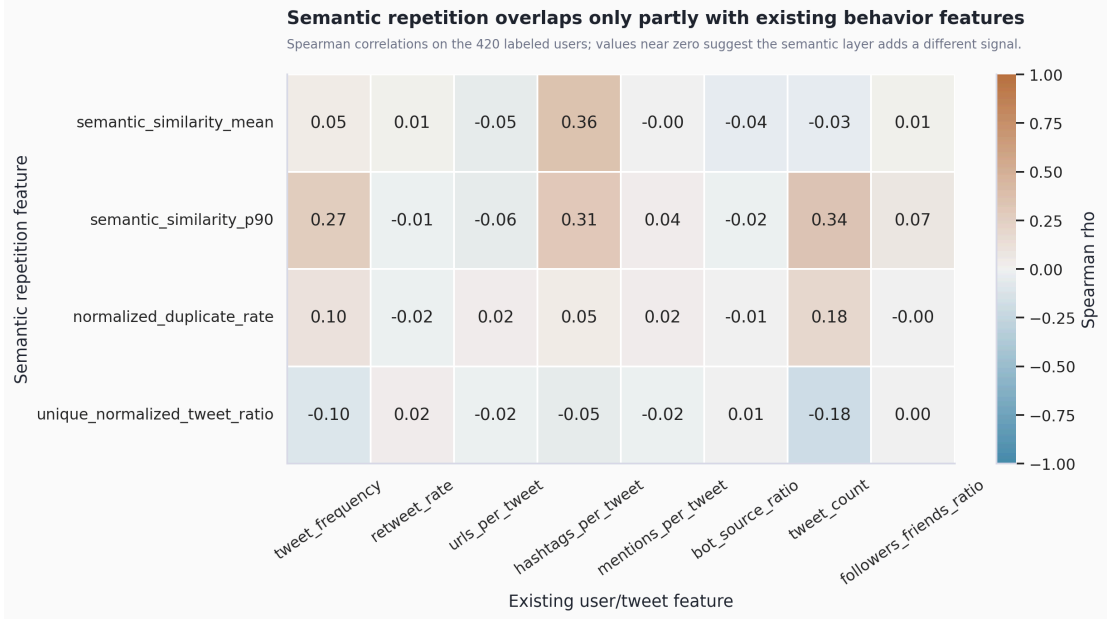


Figure 16: Corrélations entre répétition sémantique et features comportementales

Il faudrait cependant le valider sur plus que 420 profils avant d’en faire une conclusion forte. Pour une suite du projet, ce serait une piste naturelle : calculer ces features sur tout le corpus, puis vérifier si le gain se maintient sur un échantillon plus large.

8.5. Conclusion

Ce projet montre qu’une détection raisonnable de bots est possible avec des données publiques Twitter et des features assez simples. Le Random Forest donne les meilleurs résultats dans notre cadre, surtout grâce à son recall élevé. Les approches non-supervisées sont moins performantes en classification directe, mais elles confirment qu’une partie des profils suspects se regroupe naturellement dans l’espace des features.

La suite logique serait d’élargir l’annotation, puis de tester plus sérieusement les features sémantiques. Le projet reste donc un prototype, mais il donne déjà une base exploitable pour distinguer des profils humains de profils automatisés ou atypiques.

9. Bibliographie

- Benevenuto, F., Magno, G., Rodrigues, T., & Almeida, V. (2010). Detecting spammers on Twitter. **Proceedings of the 7th Annual Collaboration, Electronic Messaging, Anti-Abuse and Spam Conference (CEAS)**.
- Breiman, L. (2001). Random forests. **Machine Learning**, **45**(1), 5-32.
- Chu, Z., Gianvecchio, S., Wang, H., & Jajodia, S. (2012). Detecting automation of Twitter accounts: Are you a human, bot, or cyborg? **IEEE Transactions on Dependable and Secure Computing**, **9**(6), 811-824.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. **Machine Learning**, **20**(3), 273-297.
- Ferrara, E., Varol, O., Davis, C., Menczer, F., & Flammini, A. (2016). The rise of social bots. **Communications of the ACM**, **59**(7), 96-104.
- Lloyd, S. (1982). Least squares quantization in PCM. **IEEE Transactions on Information Theory**, **28**(2), 129-137.
- Ng, A. Y., Jordan, M. I., & Weiss, Y. (2002). On spectral clustering: Analysis and an algorithm. **Advances in Neural Information Processing Systems**, **14**.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, **12**, 2825-2830.
- Perez, C., Lemerrier, M., Birregah, B., & Corpeil, A. (2011). SPOT 1.0: Scoring Suspicious Profiles On Twitter. **Proceedings of the 2011 International Conference on Advances in Social Networks Analysis and Mining (ASONAM)**, 377-381.
- Varol, O., Ferrara, E., Davis, C. A., Menczer, F., & Flammini, A. (2017). Online human-bot interactions: Detection, estimation, and characterization. **Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM)**, 280-289.